

Если ранее не было результативного вызова `std::call_once` в отношении того же самого объекта `std::once_flag`, аргумент `func` (или его копия) вызывается словно при вызове `INVOKE(func, args)`, а вызов `std::call_once` результативен только в том случае, если вызов `func` возвращается без выдачи исключения. Если выдано исключение, оно распространяется до вызывающего кода. Если ранее уже был результативный вызов `std::call_once` в отношении того же самого объекта `std::once_flag`, вызов `std::call_once` возвращает управление без вызова `func`.

Синхронизация

Завершение результативного вызова `std::call_once` в отношении объекта `std::once_flag` «проходит до» всех последующих вызовов `std::call_once` в отношении того же самого объекта `std::once_flag`.

Выдает

Исключение `std::system_error`, когда результат не может быть достигнут, или для любого исключения, распространенного из вызова `func`.

Г.6. Заголовок `<ratio>`

Заголовок `<ratio>` предоставляет поддержку арифметических действий с рациональными числами на этапе компиляции.

Содержимое заголовка

```
namespace std
{
    template<intmax_t N, intmax_t D=1>
    class ratio;

    // ratio-арифметика
    template <class R1, class R2>
    using ratio_add = см. описание;

    template <class R1, class R2>
    using ratio_subtract = см. описание;

    template <class R1, class R2>
    using ratio_multiply = см. описание;

    template <class R1, class R2>
    using ratio_divide = см. описание;

    // ratio-сравнение
    template <class R1, class R2>
    struct ratio_equal;

    template <class R1, class R2>
    struct ratio_not_equal;
```

```

template <class R1, class R2>
struct ratio_less;

template <class R1, class R2>
struct ratio_less_equal;

template <class R1, class R2>
struct ratio_greater;

template <class R1, class R2>
struct ratio_greater_equal;

typedef ratio<1, 1000000000000000000> atto;
typedef ratio<1, 1000000000000000> femto;
typedef ratio<1, 1000000000000> pico;
typedef ratio<1, 1000000000> nano;
typedef ratio<1, 1000000> micro;
typedef ratio<1, 1000> milli;
typedef ratio<1, 100> centi;
typedef ratio<1, 10> deci;
typedef ratio<10, 1> deca;
typedef ratio<100, 1> hecto;
typedef ratio<1000, 1> kilo;
typedef ratio<1000000, 1> mega;
typedef ratio<1000000000, 1> giga;
typedef ratio<1000000000000, 1> tera;
typedef ratio<1000000000000000, 1> peta;
typedef ratio<1000000000000000000, 1> exa;
}

```

Г.6.1. Шаблон класса `std::ratio`

Шаблон класса `std::ratio` предоставляет механизм для выполнения арифметических действий на этапе компиляции, в которой используются рациональные значения, например одна вторая (`std::ratio<1,2>`), две трети (`std::ratio<2,3>`) или пятнадцать сорок третьих (`std::ratio<15,43>`). Он используется внутри стандартной библиотеки C++ для указания периода при создании экземпляра шаблона класса `std::chrono::duration`.

Определение класса

```

template <intmax_t N, intmax_t D = 1>
class ratio
{
public:
    typedef ratio<num, den> type;

    static constexpr intmax_t num= см. ниже;
    static constexpr intmax_t den= см. ниже;
};

```

Требования

D не может быть нулем.

Описание

num и den являются числителем и знаменателем дроби N/D после ее предельного сокращения. den всегда является положительным числом. Если N и D имеют одинаковые знаки, num является положительным числом; в противном случае num является отрицательным числом.

Примеры

```
ratio<4,6>::num == 2
ratio<4,6>::den == 3
ratio<4,-6>::num == -2
ratio<4,-6>::den == 3
```

Г.6.2. Псевдоним шаблона std::ratio_add

Псевдоним шаблона std::ratio_add предоставляет механизм для сложения двух значений std::ratio на этапе компиляции с применением арифметических действий с рациональными числами.

Определение

```
template <class R1, class R2>
using ratio_add = std::ratio<see below>;
```

Предусловия

R1 и R2 должны быть экземплярами шаблона класса std::ratio.

Результат

ratio_add<R1,R2> определяется в качестве псевдонима для создания экземпляра std::ratio, представляющего сумму дробей, представленных параметрами R1 и R2, если эта сумма может быть вычислена без переполнения. Если при вычислении результата возникает переполнение, программа считается некорректной. В отсутствие арифметического переполнения у std::ratio_add<R1,R2> должны быть такие же значения num и den, что и у std::ratio<R1::num * R2::den + R2::num * R1::den, R1::den * R2::den>.

Примеры

```
std::ratio_add<std::ratio<1,3>, std::ratio<2,5> >::num == 11
std::ratio_add<std::ratio<1,3>, std::ratio<2,5> >::den == 15

std::ratio_add<std::ratio<1,3>, std::ratio<7,6> >::num == 3
std::ratio_add<std::ratio<1,3>, std::ratio<7,6> >::den == 2
```

Г.6.3. Псевдоним шаблона std::ratio_subtract

Псевдоним шаблона std::ratio_subtract предоставляет механизм для вычитания двух значений std::ratio на этапе компиляции с применением арифметических действий с рациональными числами.

Определение

```
template <class R1, class R2>
using ratio_subtract = std::ratio<see below>;
```

Предусловия

R1 и R2 должны быть экземплярами шаблона класса `std::ratio`.

Результат

`ratio_subtract<R1,R2>` определяется в качестве псевдонима для создания экземпляра `std::ratio`, представляющего разность дробей, представленных параметрами R1 и R2, если эта разность может быть вычислена без переполнения. Если при вычислении результата возникает переполнение, программа считается некорректной. В отсутствие арифметического переполнения у `std::ratio_subtract<R1,R2>` должны быть такие же значения `num` и `den`, что и у `std::ratio<R1::num * R2::den - R2::num * R1::den, R1::den * R2::den>`.

Примеры

```
std::ratio_subtract<std::ratio<1,3>, std::ratio<1,5> >::num == 2
std::ratio_subtract<std::ratio<1,3>, std::ratio<1,5> >::den == 15

std::ratio_subtract<std::ratio<1,3>, std::ratio<7,6> >::num == -5
std::ratio_subtract<std::ratio<1,3>, std::ratio<7,6> >::den == 6
```

Г.6.4. Псевдоним шаблона `std::ratio_multiply`

Псевдоним шаблона `std::ratio_multiply` предоставляет механизм для умножения двух значений `std::ratio` на этапе компиляции с применением арифметических действий с рациональными числами.

Определение

```
template <class R1, class R2>
using ratio_multiply = std::ratio<see below>;
```

Предусловия

R1 и R2 должны быть экземплярами шаблона класса `std::ratio`.

Результат

`ratio_multiply<R1,R2>` определяется в качестве псевдонима для создания экземпляра `std::ratio`, представляющего произведение дробей, представленных параметрами R1 и R2, если это произведение может быть вычислено без переполнения. Если при вычислении результата возникает переполнение, программа считается некорректной. В отсутствие арифметического переполнения у `std::ratio_multiply<R1,R2>` должны быть такие же значения `num` и `den`, что и у `std::ratio<R1::num * R2::num, R1::den * R2::den>`.

Примеры

```
std::ratio_multiply<std::ratio<1,3>, std::ratio<2,5> >::num == 2
std::ratio_multiply<std::ratio<1,3>, std::ratio<2,5> >::den == 15

std::ratio_multiply<std::ratio<1,3>, std::ratio<15,7> >::num == 5
std::ratio_multiply<std::ratio<1,3>, std::ratio<15,7> >::den == 7
```

Г.6.5. Псевдоним шаблона `std::ratio_divide`

Псевдоним шаблона `std::ratio_divide` предоставляет механизм для деления двух значений `std::ratio` на этапе компиляции с применением арифметических действий с рациональными числами.

Определение

```
template <class R1, class R2>
using ratio_divide = std::ratio<see below>;
```

Предусловия

R1 и R2 должны быть экземплярами шаблона класса `std::ratio`.

Результат

`ratio_divide<R1,R2>` определяется в качестве псевдонима для создания экземпляра `std::ratio`, представляющего частное дробей, представленных параметрами R1 и R2, если это частное может быть вычислено без переполнения. Если при вычислении результата возникает переполнение, программа считается некорректной. В отсутствие арифметического переполнения у `ratio_divide<R1,R2>` должны быть такие же значения `num` и `den`, что и у `std::ratio<R1::num * R2::den, R1::den * R2::num>`.

Примеры

```
std::ratio_divide<std::ratio<1,3>, std::ratio<2,5>>>::num == 5
std::ratio_divide<std::ratio<1,3>, std::ratio<2,5>>>::den == 6

std::ratio_divide<std::ratio<1,3>, std::ratio<15,7>>>::num == 7
std::ratio_divide<std::ratio<1,3>, std::ratio<15,7>>>::den == 45
```

Г.6.6. Шаблон класса `std::ratio_equal`

Псевдоним шаблона `std::ratio_equal` предоставляет механизм для сравнения на равенство двух значений `std::ratio` на этапе компиляции с применением арифметических действий с рациональными числами.

Определение класса

```
template <class R1, class R2>
class ratio_equal:
    public std::integral_constant<
        bool, (R1::num == R2::num) && (R1::den == R2::den)>
{
};
```

Предусловия

R1 и R2 должны быть экземплярами шаблона класса `std::ratio`.

Примеры

```
std::ratio_equal<std::ratio<1,3>, std::ratio<2,6>>>::value == true
std::ratio_equal<std::ratio<1,3>, std::ratio<1,6>>>::value == false
std::ratio_equal<std::ratio<1,3>, std::ratio<2,3>>>::value == false
std::ratio_equal<std::ratio<1,3>, std::ratio<1,3>>>::value == true
```

Г.6.7. Шаблон класса `std::ratio_not_equal`

Псевдоним шаблона `std::ratio_not_equal` предоставляет механизм для сравнения на неравенство двух значений `std::ratio` на этапе компиляции с применением арифметических действий с рациональными числами.

Определение класса

```
template <class R1, class R2>
class ratio_not_equal:
    public std::integral_constant<bool,!ratio_equal<R1,R2>::value>
{};
```

Предусловия

R1 и R2 должны быть экземплярами шаблона класса `std::ratio`.

Примеры

```
std::ratio_not_equal<std::ratio<1,3>, std::ratio<2,6> >::value == false
std::ratio_not_equal<std::ratio<1,3>, std::ratio<1,6> >::value == true
std::ratio_not_equal<std::ratio<1,3>, std::ratio<2,3> >::value == true
std::ratio_not_equal<std::ratio<1,3>, std::ratio<1,3> >::value == false
```

Г.6.8. Шаблон класса `std::ratio_less`

Псевдоним шаблона `std::ratio_less` предоставляет механизм для сравнения двух значений `std::ratio` на этапе компиляции с применением арифметических действий с рациональными числами.

Определение класса

```
template <class R1, class R2>
class ratio_less:
    public std::integral_constant<bool,see below>
{};
```

Предусловия

R1 и R2 должны быть экземплярами шаблона класса `std::ratio`.

Результат

`std::ratio_less<R1,R2>` выводится из шаблона `std::integral_constant<bool,value>`, где `value` — это $(R1::num * R2::den) < (R2::num * R1::den)$. По возможности в реализации должен использоваться метод вычисления результата, избегающий переполнения. Если возникает переполнение, программа считается некорректной.

Примеры

```
std::ratio_less<std::ratio<1,3>, std::ratio<2,6> >::value == false
std::ratio_less<std::ratio<1,6>, std::ratio<1,3> >::value == true
std::ratio_less<
    std::ratio<999999999,1000000000>,
    std::ratio<1000000001,1000000000> >::value == true
std::ratio_less<
    std::ratio<1000000001,1000000000>,
    std::ratio<999999999,1000000000> >::value == false
```


Г.6.9. Шаблон класса `std::ratio_greater`

Псевдоним шаблона `std::ratio_greater` предоставляет механизм для сравнения двух значений `std::ratio` на этапе компиляции с применением арифметических действий с рациональными числами.

Определение класса

```
template <class R1, class R2>
class ratio_greater:
    public std::integral_constant<bool, ratio_less<R2, R1>::value>
{};
```

Предусловия

R1 и R2 должны быть экземплярами шаблона класса `std::ratio`.

Г.6.10. Шаблон класса `std::ratio_less_equal`

Псевдоним шаблона `std::ratio_less_equal` предоставляет механизм для сравнения двух значений `std::ratio` на этапе компиляции с применением арифметических действий с рациональными числами.

Определение класса

```
template <class R1, class R2>
class ratio_less_equal:
    public std::integral_constant<bool, !ratio_less<R2, R1>::value>
{};
```

Предусловия

R1 и R2 должны быть экземплярами шаблона класса `std::ratio`.

Г.6.11. Шаблон класса `std::ratio_greater_equal`

Псевдоним шаблона `std::ratio_greater_equal` предоставляет механизм для сравнения двух значений `std::ratio` на этапе компиляции с применением арифметических действий с рациональными числами.

Определение класса

```
template <class R1, class R2>
class ratio_greater_equal:
    public std::integral_constant<bool, !ratio_less<R1, R2>::value>
{};
```

Предусловия

R1 и R2 должны быть экземплярами шаблона класса `std::ratio`.

Г.7. Заголовок `<thread>`

Заголовок `<thread>` предоставляет средства управления потоками и идентификации потоков, а также функции для ввода текущего потока в спящий режим.

Содержимое заголовка

```
namespace std
{
    class thread;

    namespace this_thread
    {
        thread::id get_id() noexcept;

        void yield() noexcept;

        template<typename Rep,typename Period>
        void sleep_for(
            std::chrono::duration<Rep,Period> sleep_duration);

        template<typename Clock,typename Duration>
        void sleep_until(
            std::chrono::time_point<Clock,Duration> wake_time);
    }
}
```

Г.7.1. Класс std::thread

Класс `std::thread` используется для управления потоком выполнения. Он предоставляет средства для запуска нового потока выполнения и ожидания завершения потока выполнения. Он также предоставляет средства для идентификации и другие функции для управления потоками выполнения.

Определение класса

```
class thread
{
public:
    // Типы
    class id;
    typedef implementation-defined native_handle_type; // необязательно

    // Конструкторы и деструкторы
    thread() noexcept;

    ~thread();

    template<typename Callable,typename Args...>
    explicit thread(Callable&& func,Args&&... args);

    // Копирование и перемещение
    thread(thread const& other) = delete;
    thread(thread&& other) noexcept;

    thread& operator=(thread const& other) = delete;
    thread& operator=(thread&& other) noexcept;

    void swap(thread& other) noexcept;
```



```

void join();
void detach();
bool joinable() const noexcept;

id get_id() const noexcept;

native_handle_type native_handle();

static unsigned hardware_concurrency() noexcept;
};

void swap(thread& lhs, thread& rhs);

```

Класс `std::thread::id`

Экземпляр `std::thread::id` идентифицирует конкретный поток выполнения.

Определение класса

```

class thread::id
{
public:
    id() noexcept;

};

bool operator==(thread::id x, thread::id y) noexcept;
bool operator!=(thread::id x, thread::id y) noexcept;
bool operator<(thread::id x, thread::id y) noexcept;
bool operator<=(thread::id x, thread::id y) noexcept;
bool operator>(thread::id x, thread::id y) noexcept;
bool operator>=(thread::id x, thread::id y) noexcept;

template<typename charT, typename traits>
basic_ostream<charT, traits>&
operator<< (basic_ostream<charT, traits>&& out, thread::id id);

```

Примечания

Значение `std::thread::id`, идентифицирующее конкретный поток выполнения, должно отличаться от значения изначально созданного экземпляра `std::thread::id` и от любого значения, представляющего другой поток выполнения.

Значение `std::thread::id` для конкретного потока непредсказуемо и может варьироваться между выполнениями одной и той же программы.

Объект `std::thread::id` является `CopyConstructible` и `CopyAssignable`, поэтому экземпляры `std::thread::id` можно свободно копировать и присваивать.

Конструктор по умолчанию `std::thread::id`

Создает объект `std::thread::id`, не представляющий какой-либо поток выполнения.

Объявление

```
id() noexcept;
```

Результат

Создает экземпляр `std::thread::id`, имеющий особое значение, *не относящееся ни к какому потоку*.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Все изначально созданные экземпляры `std::thread::id` хранят одно и то же значение.

Оператор сравнения для определения равенства `std::thread::id`

Сравнивает два экземпляра `std::thread::id`, чтобы определить, не представляют ли они один и тот же поток выполнения.

Объявление

```
bool operator==(std::thread::id lhs, std::thread::id rhs) noexcept;
```

Возвращает

`true`, если и `lhs`, и `rhs` представляют один и тот же поток выполнения или оба они имеют особое значение, *не имеющее отношения ни к одному* из потоков. `false`, если `lhs` и `rhs` представляют разные потоки выполнения, или один представляет поток выполнения, а другой имеет особое значение, *не имеющее отношения ни к одному* из потоков.

Выдает

Ничего не выдает.

Оператор сравнения для определения неравенства `std::thread::id`

Сравнивает два экземпляра `std::thread::id`, чтобы определить, не представляют ли они разные потоки выполнения.

Объявление

```
bool operator!=(std::thread::id lhs, std::thread::id rhs) noexcept;
```

Возвращает

```
!(lhs==rhs)
```

Выдает

Ничего не выдает.

Оператор сравнения для определения соотношения «меньше» `std::thread::id`

Сравнивает два экземпляра `std::thread::id`, чтобы определить, не находится ли один перед другим в общем порядке значений идентификаторов потоков.

Объявление

```
bool operator<(std::thread::id lhs, std::thread::id rhs) noexcept;
```

Возвращает

`true`, если значение `lhs` оказывается до значения `rhs` в общем порядке значений идентификаторов потоков. Если `lhs != rhs`, то одно из вычислений, `lhs < rhs` или `rhs < lhs`, возвращает `true`, а другое возвращает `false`. Если `lhs == rhs`, оба вычисления, `lhs < rhs` и `rhs < lhs`, возвращают `false`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Особое значение, не имеющее отношения ни к одному из потоков, которое хранится в изначально созданном экземпляре `std::thread::id`, в сравнении оказывается меньше значения любого экземпляра `std::thread::id`, представляющего поток выполнения. Если два экземпляра `std::thread::id` равны, то ни один из них не меньше другого. Любой набор различных значений `std::thread::id` формирует общий порядок, остающийся постоянным на всем протяжении выполнения программы. Между выполнениями одной и той же программы этот порядок может варьироваться.

Оператор сравнения для определения соотношения «меньше или равно» `std::thread::id`

Сравнивает два экземпляра `std::thread::id`, чтобы определить, не находится ли один перед другим в общем порядке значений идентификаторов потоков или не равен ли он ему.

Объявление

```
bool operator<=(std::thread::id lhs, std::thread::id rhs) noexcept;
```

Возвращает

```
!(rhs < lhs)
```

Выдает

Ничего не выдает.

Оператор сравнения для определения соотношения «больше» `std::thread::id`

Сравнивает два экземпляра `std::thread::id`, чтобы определить, не находится ли один после другого в общем порядке значений идентификаторов потоков.

Объявление

```
bool operator>(std::thread::id lhs, std::thread::id rhs) noexcept;
```

Возвращает

```
rhs < lhs
```

Выдает

Ничего не выдает.

Оператор сравнения для определения соотношения «больше или равно» `std::thread::id`

Сравнивает два экземпляра `std::thread::id`, чтобы определить, не находится ли один после другого в общем порядке значений идентификаторов потоков или не равен ли он ему.

Объявление

```
bool operator>=(std::thread::id lhs, std::thread::id rhs) noexcept;
```

Возвращает

```
!(lhs<rhs)
```

Выдает

Ничего не выдает.

Оператор вставки в поток `std::thread::id`

Записывает строку представления значения `std::thread::id` в указанный поток.

Объявление

```
template<typename charT, typename traits>  
basic_ostream<charT, traits>&  
operator<< (basic_ostream<charT, traits>&& out, thread::id id);
```

Результат

Вставляет строку представления значения `std::thread::id` в указанный поток.

Возвращает

```
out
```

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Формат строкового представления точно не определен. Экземпляры `std::thread::id`, сравниваемые на равенство, имеют одинаковые представления, а экземпляры, не равные друг другу, имеют различные представления.

Псевдоним типа `std::thread::native_handle_type`

`native_handle_type` является псевдонимом типа, которым можно воспользоваться вместе с API конкретной платформы.

Объявление

```
typedef implementation-defined native_handle_type;
```

ПРИМЕЧАНИЕ

Этот псевдоним типа носит необязательный характер. Если он присутствует, реализация должна предоставить тип, подходящий для использования с исходными API конкретной платформы.

Компонентная функция `std::thread::native_handle`

Возвращает значение типа `native_handle_type`, представляющее поток выполнения, связанный с `*this`.

Объявление

```
native_handle_type native_handle();
```

ПРИМЕЧАНИЕ

Эта функция носит необязательный характер. Если она присутствует, то возвращаемое значение должно быть подходящим для использования с исходными API конкретной платформы.

Конструктор по умолчанию `std::thread`

Создает объект `std::thread`, не связанный с потоком выполнения.

Объявление

```
thread() noexcept;
```

Результат

Создает экземпляр `std::thread`, не имеющий связанного с ним потока выполнения.

Постусловия

Для только что созданного объекта `std::thread`, `x`, `x.get_id()==id()`.

Выдает

Ничего не выдает.

Конструктор `std::thread`

Создает объект `std::thread`, связанный с новым потоком выполнения.

Объявление

```
template<typename Callable,typename Args...>
explicit thread(Callable&& func,Args&&... args);
```

Предусловия

`func` и каждый элемент `args` должны быть `MoveConstructible`.

Результат

Создает экземпляр `std::thread` и связывает его с только что созданным потоком выполнения. Копирует или перемещает `func` и каждый элемент `args` во внутреннее

хранилище, существующее на протяжении всего жизненного цикла нового потока выполнения. Выполняет `INVOKE(copy-of-func, copy-of-args)` в отношении нового потока выполнения.

Постусловия

Для только что созданного объекта `std::thread, x`, `x.get_id()!=id()`.

Выдает

Исключение типа `std::system_error`, если не может запустить новый поток. Любое исключение, выданное при копировании `func` или `args` во внутреннее хранилище.

Синхронизация

Вызов конструктора «проходит до» выполнения предоставленной функции, выполняемой в отношении только что созданного потока выполнения.

Перемещающий конструктор `std::thread`

Перемещает принадлежность потока выполнения из одного объекта `std::thread` в только что созданный объект `std::thread`.

Объявление

```
thread(thread&& other) noexcept;
```

Результат

Создает экземпляр `std::thread`. Если до вызова конструктора у `other` имелся связанный с ним поток выполнения, то теперь этот поток связан с только что созданным объектом `std::thread`. В противном случае только что созданный объект `std::thread` не имеет связанного с ним потока выполнения.

Постусловия

Для только что созданного объекта `std::thread, x`, `x.get_id()` равно значению `other.get_id()`, имевшемуся до вызова конструктора. `other.get_id()==id()`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Объекты `std::thread` не являются `CopyConstructible`, поэтому копирующий конструктор отсутствует и есть только этот перемещающий конструктор.

Деструктор `std::thread`

Уничтожает объект `std::thread`.

Объявление

```
~thread();
```


Результат

Уничтожает `*this`. Если у `*this` имеется связанный с ним поток выполнения (в результате выполнения `this->joinable()` будет возвращено значение `true`), вызывает `std::terminate()` для прекращения выполнения программы.

Выдает

Ничего не выдает.

Перемещающий оператор присваивания `std::thread`

Перемещает принадлежность потока выполнения из одного объекта `std::thread` в другой объект `std::thread`.

Объявление

```
thread& operator=(thread&& other) noexcept;
```

Результат

Если до вызова `this->joinable()` возвращает `true`, вызывает `std::terminate()` для прекращения выполнения программы. Если до присваивания у `other` имелся связанный с ним поток выполнения, то теперь этот поток выполнения связан с `*this`. В противном случае у `*this` нет связанного с ним потока выполнения.

Постусловия

До вызова значение `this->get_id()` равно значению `other.get_id()`. `other.get_id()==id()`.

Выдает

Ничего не выдает.

ПРИМЕЧАНИЕ

Объекты `std::thread` не являются `CopyAssignable`, поэтому копирующий оператор присваивания отсутствует и имеется только перемещающий оператор присваивания.

Компонентная функция `std::thread::swap`

Производит обмен принадлежностью связанных потоков выполнения между двумя объектами `std::thread`.

Объявление

```
void swap(thread& other) noexcept;
```

Результат

Если до вызова у `other` имелся связанный с ним поток выполнения, то теперь этот поток связан с `*this`. В противном случае у `*this` нет связанного с ним потока выполнения. Если до вызова у `*this` имелся связанный с ним поток выполнения, то теперь этот поток связан с `other`. В ином случае у `other` нет связанного с ним потока выполнения.

Постусловия

До вызова значение `this->get_id()` равно значению `other.get_id()`. До вызова значение `other.get_id()` равно значению `this->get_id()`.

Выдает

Ничего не выдает.

Некомпонентная функция обмена для `std::threads`

Производит обмен принадлежности связанных потоков выполнения между двумя объектами `std::thread`.

Объявление

```
void swap(thread& lhs, thread& rhs) noexcept;
```

Результат

```
lhs.swap(rhs)
```

Выдает

Ничего не выдает.

Компонентная функция `std::thread::joinable`

Выполняет запрос, имеется ли у `*this` связанный с ним поток выполнения.

Объявление

```
bool joinable() const noexcept;
```

Возвращает

`true`, если у `*this` имеется связанный с ним поток выполнения, или `false` в противном случае.

Выдает

Ничего не выдает.

Компонентная функция `std::thread::join`

Ожидает завершения потока выполнения, связанного с `*this`.

Объявление

```
void join();
```

Предусловия

Функция `this->joinable()` возвращает `true`.

Результат

Блокирует текущий поток до завершения потока выполнения, связанного с `*this`.

Постусловия

`this->get_id()==id()`. Поток выполнения, связанный с `*this` до вызова, завершился.

Синхронизация

Завершение потока выполнения, связанного с `*this` до вызова, «происходит до» возвращения из вызова `join()`.

Выдает

Исключение `std::system_error`, если результаты не могут быть достигнуты, или `this->joinable()` возвращает `false`.

Компонентная функция `std::thread::detach`

Отсоединяет поток выполнения, связанный с `*this` для завершения.

Объявление

```
void detach();
```

Предусловия

Функция `this->joinable()` возвращает `true`.

Результат

Отсоединяет поток выполнения, связанный с `*this`.

Постусловия

`this->get_id()==id(), this->joinable()==false`. Поток выполнения, связанный с `*this` до вызова, отсоединяется и больше не связан с объектом `std::thread`.

Выдает

Исключение `std::system_error`, если результаты не могут быть достигнуты, или при вызове `this->joinable()` возвращает `false`.

Компонентная функция `std::thread::get_id`

Возвращает значение типа `std::thread::id`, которое идентифицирует поток выполнения, связанный с `*this`.

Объявление

```
thread::id get_id() const noexcept;
```

Возвращает

Если у `*this` имеется связанный с ним поток выполнения, возвращает экземпляр `std::thread::id`, идентифицирующий этот поток. В противном случае возвращает изначально созданный объект `std::thread::id`.

Выдает

Ничего не выдает.

Статическая компонентная функция `std::thread::hardware_concurrency`

Возвращает подсказку о количестве потоков, способных конкурентно запускаться на текущем оборудовании.

Объявление

```
unsigned hardware_concurrency() noexcept;
```

Возвращает

Количество потоков, способных конкурентно запускаться на текущем оборудовании. Это, к примеру, может быть число процессоров, имеющихся в системе. Если такая информация недоступна или не поддается точному определению, эта функция возвращает 0.

Выдает

Ничего не выдает.

Г.7.2. Пространство имен `this_thread`

Функции в пространстве имен `std::this_thread` работают с вызывающим потоком.

Некомпонентная функция `std::this_thread::get_id`

Возвращает значение типа `std::thread::id`, идентифицирующее текущий поток выполнения.

Объявление

```
thread::id get_id() noexcept;
```

Возвращает

Экземпляр `std::thread::id`, идентифицирующий текущий поток.

Выдает

Ничего не выдает.

Некомпонентная функция `std::this_thread::yield`

Используется для информирования библиотеки о том, что поток, вызвавший функцию, не нуждается в выполнении в точке вызова. Зачастую используется в коротких циклах во избежание потребления излишнего времени центрального процессора.

Объявление

```
void yield() noexcept;
```

Результат

Предоставляет библиотеке возможность спланировать вместо текущего потока что-либо другое.

Выдает

Ничего не выдает.

Некомпонентная функция `std::this_thread::sleep_for`

Препоставляет выполнение текущего потока на указанный период времени.

Объявление

```
template<typename Rep, typename Period>  
void sleep_for(std::chrono::duration<Rep, Period> const& relative_time);
```

Результат

Блокирует текущий поток до истечения времени, заданного параметром `relative_time`.

ПРИМЕЧАНИЕ

Поток может быть заблокирован на время, дольше указанного. По возможности истекшее время нужно измерять по стабильным часам.

Выдает

Ничего не выдает.

Некомпонентная функция `std::this_thread::sleep_until`

Приостанавливает выполнение текущего потока до достижения заданного момента времени.

Объявление

```
template<typename Clock, typename Duration>
void sleep_until(
    std::chrono::time_point<Clock, Duration> const& absolute_time);
```

Результат

Блокирует текущий поток до достижения указанного на часах `Clock` момента времени, заданного параметром `absolute_time`.

ПРИМЕЧАНИЕ

По продолжительности блокировки вызывающего потока не дается никаких гарантий, гарантируется только то, что на момент разблокировки потока функцией `Clock::now()` будет возвращено время, равное времени, заданному параметром `absolute_time`, или более позднее.

Выдает

Ничего не выдает.